

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115** Bug **15** Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name...

Class names should comply with a naming convention

Code Smell

Track uses of "TODO" tags

Code Smell

Deprecated code should be removed

Code Smell

Cyclomatic Complexity of functions should not be too high

Code Smell

Constant names should comply with a naming convention

Code Smell

Unnecessary nullable in final declaration should be removed

Code Smell

Dependencies should be sorted

Code Smell

Unused assignments should be removed

Code Smell

Standard outputs should not be used directly to log anything

Code Smell

Variables should not be initialized with "null"

Code Smell

Files should end with a newline

Code Smell

Unnecessary nullable in final declaration should be removed

Analyze your code

Intentionality - Clear Maintainability

Code Smell Major

A `final` or `const` variable that is declared as a nullable type, but then initialized with a non-null value, should rather be declared as a non-nullable type.

Why is this an issue? How can I fix it? More Info

Unlike `var` variables, `final` and `const` variables need to be initialized inline.

```
final int i = 42;
const int i = 42;
```

An exception to the inline initialization is when the late keyword is used, which allows the variable to be initialized later, and that can only apply to non-`const` variables:

```
late final int i1;
late int? i2;

void main() {
  // ...
  i1 = int.parse(String.fromEnvironment('I1'));
  i2 = int.parse(String.fromEnvironment('I2'));
  // Use i
}
```

Aforementioned exceptions apart, if:

- the declared type is explicit (e.g. `final int?` or `const int?`)
- the declared type is a nullable type (e.g. `int?`)
- the inline initialization value is not null (e.g. `42`)

then the declared type could be non-nullable instead, because the value of the variable will never be `null` under any circumstances, due to the `final` or `const` constraints imposed.

The rule applies to:

- local variables of functions and methods
- top-level variables

What is the potential impact?

Because Dart 3 enforces **sound null safety**, not following this rule may have multiple consequences.

Code complexity

It may make the code **unnecessarily complicated**, as the developer will have to handle the `null` case even though it will never happen: for example, if the variable `t` is of type `T?`, any operation `f` on `t` will have to:

- either use the null-aware operator `?`: e.g. `t?.f()`
- or the null assertion operator `!` on `t`: e.g. `t!.f()`
- or narrow the type from `T?` to `T` with an `if` statement, a conditional expression, or via pattern matching: e.g. `t != null ? t.f() : ...`

Neither of these options is necessary if the variable is declared as `final T` or `const T` instead.

Unclear intent

If the variable is declared as `final T?` or `const T?`, and used far from its declaration, the `?` can be misleading, since the developer may assume that the variable can be `null` at some point in its lifetime, even though it can't.

Exceptions

The rule does not apply to class instance fields, whether they are `late` or not.

```
class A {
  final int? i = 42; // Non applicable
  A(this.a);
}
```

However, it does apply to class static fields:

```
class A {
  static final int? i1 = 42; // Non compliant
  static const int? i2 = 42; // Non compliant
}
```

Available In:

sonarcloud

